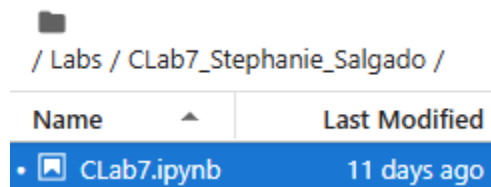


CLab 7

Stephanie Salgado

Setup



I can go straight into JupyterHub since I've already set it up in a past lab. Once there, I create a folder where I will include all materials for this lab.

Configuring environment

Configure environment

```
from fabrictestbed_extensions.fablib.fablib import FablibManager as fablib_manager
fablib = fablib_manager()
conf = fablib.show_config()
```

FABlib Config

Orchestrator	orchestrator.fabric-testbed.net
Credential Manager	cm.fabric-testbed.net
Core API	uis.fabric-testbed.net
Token File	/home/fabric/.tokens.json
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
Bastion Host	bastion.fabric-testbed.net
Bastion Username	ssalg03_0000228084
Bastion Private Key File	/home/fabric/work/fabric_config/fabric-bastion-key
Slice Public Key File	/home/fabric/work/fabric_config/slice_key.pub
Slice Private Key File	/home/fabric/work/fabric_config/slice_key
Sites to avoid	
SSH Command Line	ssh -i {{ _self_private_ssh_key_file }} -F /home/fabric/work/fabric_config/ssh_config {{ _self_username }}@{{ _self_management_ip }}
Log Level	INFO
Log File	/tmp/fablib/fablib.log
Bastion SSH Config File	/home/fabric/work/fabric_config/ssh_config
Version	1.7.3
Data directory	/tmp/fablib

Using "CLab7.ipynb", I run the cell to configure the environment.

Defining configuration

Define configuration for this experiment (two hosts and router in line topology)

```
slice_name="Wireshark_Stephanie_Salgado" + fablib.get_bastion_username()

node_conf = [
    {'name': "romeo", 'cores': 2, 'ram': 4, 'disk': 10, 'image': 'default_ubuntu_22', 'packages': []},
    {'name': "juliet", 'cores': 2, 'ram': 4, 'disk': 10, 'image': 'default_ubuntu_22', 'packages': []},
    {'name': "hamlet", 'cores': 2, 'ram': 4, 'disk': 10, 'image': 'default_ubuntu_22', 'packages': []},
]
net_conf = [
    {"name": "net0", "subnet": "10.0.0.0/24", "nodes": [{"name": "romeo", "addr": "10.0.0.100"}, {"name": "juliet", "addr": "10.0.0.101"}, {"name": "hamlet", "addr": "10.0.0.102"}]}
]
route_conf = [ ]
exp_conf = {'cores': sum([ n['cores'] for n in node_conf]), 'nic': sum([len(n['nodes']) for n in net_conf]) }
```

For this experiment there will be two hosts and a router.

Reserving resources

```
try:
    slice = fablib.get_slice(slice_name)
    print("You already have a slice by this name!")
    print("If you previously reserved resources, skip to the 'log in to resources' section.")
except:
    print("You don't have a slice named %s yet." % slice_name)
    print("Continue to the next step to make one.")
    slice = fablib.new_slice(name=slice_name)
```

You don't have a slice named Wireshark_Stephanie_Salgadossalg03_0000228084 yet.
Continue to the next step to make one.

We will select a random site that has sufficient resources for our experiment:

```
#while True:
#    site_name = fablib.get_random_site()
#    if ( (fablib.resources.get_core_available(site_name) > 1.2*exp_conf['cores']) and
#        (fablib.resources.get_component_available(site_name, 'SharedNIC-ConnectX-6') > 1.2*exp_conf['nic']) ):
#        break
site_name = "EDUKY"
print(site_name)
fablib.show_site(site_name)
```

EDUKY

Site

Name	EDUKY
State	Active
Address	301 Hilltop Avenue, Lexington, KY 40506
Location	(38.0325, -84.502801)
PTP Capable	True
Hosts	18
CPUs	36

First, we check if the slice already exists. Since it doesn't, I can proceed to reserve resources from any random site with sufficient resources. In this case, "EDUKY" had sufficient resources for this experiment.

Adding hosts and network segments

```
# this cell sets up the nodes
for n in node_conf:
    slice.add_node(name=n['name'], site=site_name,
                  cores=n['cores'],
                  ram=n['ram'],
                  disk=n['disk'],
                  image=n['image'])

# this cell sets up the network segments
for n in net_conf:
    ifaces = [slice.get_node(node["name"]).add_component(model="NIC_Basic",
                                                          name=n["name"]).get_interfaces()[0] for node in n['nodes']]
    slice.add_l2network(name=n["name"], type='L2Bridge', interfaces=ifaces)
```

I used the cells above to set up the nodes and network segments.

Submitting and viewing the slice

```
slice.submit()
```

Retry: 9, Time: 233 sec

Slice

ID	75d4395b-e43e-4b77-ac88-a9de517b0738
Name	Wireshark_Stephanie_Salgadossalg03_0000228084
Lease Expiration (UTC)	2024-10-30 17:42:07 +0000
Lease Start (UTC)	2024-10-29 17:42:07 +0000
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
State	StableOK

ID	Name	Cores	RAM	Disk	Image	Image Type	Host
7d52b4b6-4289-4bf9-98c8-933ff47cfd6f	hamlet	2	4	10	default_ubuntu_22	qcow2	eduky-w15.fabric-testbed.net
2ad21efd-6d25-4399-921b-db85f8cdd2e5	juliet	2	4	10	default_ubuntu_22	qcow2	eduky-w14.fabric-testbed.net
6700e903-305c-473c-93d0-bab073ec03c7	romeo	2	4	10	default_ubuntu_22	qcow2	eduky-w17.fabric-testbed.net

Networks

ID	Name	Layer	Type	Site	Subnet	Gateway	State	Error
eec22757-66bd-40b2-a0cd-2c84cef6a73d	net0	L2	L2Bridge	EDUKY	None	None	Active	

```
slice.get_state()
slice.wait_ssh(progress=True)
```

```
Waiting for slice . Slice state: StableOK
Waiting for ssh in slice . ssh successful
True
```

Once everything was configured, I was able to submit and check the slice.

Configuring resources

With the slice ready, I can begin configuring all the resources.

Installing packages

```
# install packages
# this will take a while and will run in background while you do other steps
for n in node_conf:
    if len(n['packages']):
        node = slice.get_node(n['name'])
        pkg = " ".join(n['packages'])
        node.execute_thread("sudo apt update; sudo apt -y install %s" % pkg)
```

First, I have to install the necessary packages.

Bringing up interfaces

```
# bring interfaces up and either assign an address (if there is one) or flush address
from ipaddress import ip_address, IPv4Address, IPv4Network

for net in net_conf:
    for n in net['nodes']:
        if_name = n['name'] + '-' + net['name'] + '-p1'
        iface = slice.get_interface(if_name)
        iface.ip_link_up()
        if n['addr']:
            iface.ip_addr_add(addr=n['addr'], subnet=IPv4Network(net['subnet']))
        else:
            iface.get_node().execute("sudo ip addr flush dev %s" % iface.get_device_name())
```

Next, I bring up the interfaces and assign an address or flush address.

“hosts” file

```
# prepare a "hosts" file that has names and addresses of every node
hosts_txt = [ "%s\t%s" % ( n['addr'], n['name'] ) for net in net_conf for n in net['nodes'] if type(n) is dict and n['addr']]
for n in slice.get_nodes():
    for h in hosts_txt:
        n.execute("echo %s | sudo tee -a /etc/hosts" % h)

10.0.0.100 romeo
10.0.0.101 juliet
10.0.0.102 hamlet
10.0.0.100 romeo
10.0.0.101 juliet
10.0.0.102 hamlet
10.0.0.100 romeo
10.0.0.101 juliet
10.0.0.102 hamlet
```

With that done, the cell above shows the names and addresses of each node.

Enabling IPv4 forwarding

```
# enable IPv4 forwarding on all nodes
for n in slice.get_nodes():
    n.execute("sudo sysctl -w net.ipv4.ip_forward=1")

net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1
```

Then, I can enable forwarding on all nodes.

Setting up static routes

```
# set up static routes
for rt in route_conf:
    for n in rt['nodes']:
        slice.get_node(name=n).ip_route_add(subnet=IPv4Network(rt['addr']), gateway=rt['gw'])
```

Now, with all packages installed and all the interfaces being brought up and assigned an address (if there was one available) or a flush address, I can set up the static routes.

Turning off segmentation offload on interfaces

```
# turn off segmentation offload on interfaces
for iface in slice.get_interfaces():
    iface_name = iface.get_device_name()
    n = iface.get_node()
    offloads = ["gro", "lro", "gso", "tso"]
    for offload in offloads:
        n.execute("sudo ethtool -K %s %s off" % (iface_name, offload))
```

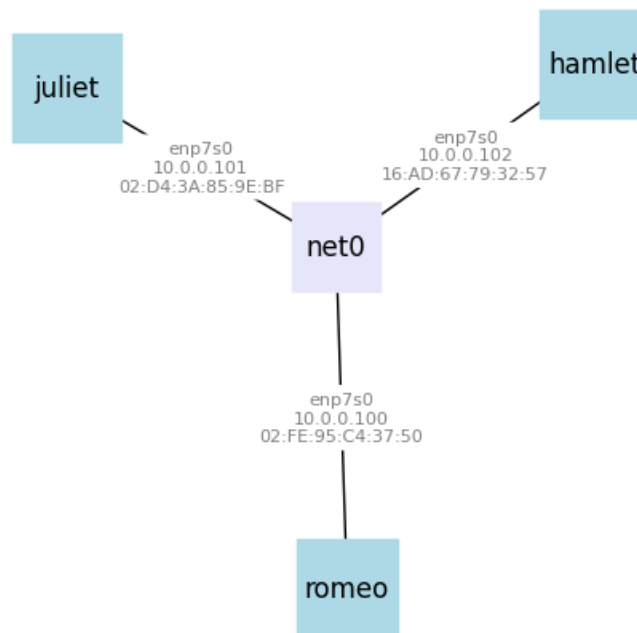
Finally, I can turn off segmentation offload on the interfaces.

Drawing the network topology

```
l2_nets = [(n.get_name(), {'color': 'lavender'}) for n in slice.get_l2networks() ]
l3_nets = [(n.get_name(), {'color': 'pink'}) for n in slice.get_l3networks() ]
hosts = [(n.get_name(), {'color': 'lightblue'}) for n in slice.get_nodes()]
nodes = l2_nets + l3_nets + hosts
ifaces = [iface.toDict() for iface in slice.get_interfaces()]
edges = [(iface['network'], iface['node'],
          {'label': iface['physical_dev'] + '\n' + iface['ip_addr'] + '\n' + iface['mac']}) for iface in ifaces]

import networkx as nx
import matplotlib.pyplot as plt
plt.figure(figsize=(len(nodes),len(nodes)))
G = nx.Graph()
G.add_nodes_from(nodes)
G.add_edges_from(edges)
pos = nx.spring_layout(G)
nx.draw(G, pos, node_shape='s',
        node_color=[n[1]['color'] for n in nodes],
        node_size=[len(n[0])*400 for n in nodes],
        with_labels=True);
nx.draw_networkx_edge_labels(G,pos,
                             edge_labels=nx.get_edge_attributes(G,'label'),
                             font_color='gray', font_size=8, rotate=False);
```

I run the cells that are responsible for drawing the network topology.



This is the topology for the network I will use for this experiment.

Getting SSH commands

```
import pandas as pd
pd.set_option('display.max_colwidth', None)
slice_info = [{ 'Name': n.get_name(), 'SSH command': n.get_ssh_command() } for n in slice.get_nodes()]
pd.DataFrame(slice_info).set_index('Name')
```

SSH command

Name	
romeo	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe07:222d
juliet	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe99:3455
hamlet	ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe2c:e508

After running the cell, I have the SSH commands I need for each node.

Logging into each node

romeo:

```
fabric@spring:Clab7_Stephanie_Salgado-16%$ ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe07:222d
Warning: Permanently added 'bastion.fabric-testbed.net' (ED25519) to the list of known hosts.
Warning: Permanently added '2610:1e0:1700:206:f816:3eff:fe07:222d' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-112-generic x86_64)
```

```
Last login: Tue Oct 29 17:44:02 2024 from 2610:1e0:1700:205::51
ubuntu@romeo:~$
```

juliet:

```
fabric@spring:Clab7_Stephanie_Salgado-16%$ ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe99:3455
Warning: Permanently added 'bastion.fabric-testbed.net' (ED25519) to the list of known hosts.
Warning: Permanently added '2610:1e0:1700:206:f816:3eff:fe99:3455' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-112-generic x86_64)
```

```
Last login: Tue Oct 29 17:44:02 2024 from 2610:1e0:1700:205::51
ubuntu@juliet:~$
```

hamlet:

```
fabric@spring:Clab7_Stephanie_Salgado-16%$ ssh -i /home/fabric/work/fabric_config/slice_key -F /home/fabric/work/fabric_config/ssh_config ubuntu@2610:1e0:1700:206:f816:3eff:fe2c:e508
Warning: Permanently added 'bastion.fabric-testbed.net' (ED25519) to the list of known hosts.
Warning: Permanently added '2610:1e0:1700:206:f816:3eff:fe2c:e508' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-112-generic x86_64)
```

```
Last login: Tue Oct 29 17:44:03 2024 from 2610:1e0:1700:205::51
ubuntu@hamlet:~$
```

Basic ARP request and response

Identifying the experiment interface

```
ubuntu@romeo:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:07:22:2d brd ff:ff:ff:ff:ff:ff
    inet 10.30.9.163/19 metric 100 brd 10.30.31.255 scope global dynamic enp3s0
        valid_lft 84196sec preferred_lft 84196sec
    inet6 2610:1e0:1700:206:f816:3eff:fe07:222d/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86347sec preferred_lft 14347sec
    inet6 fe80::f816:3eff:fe07:222d/64 scope link
        valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 02:fe:95:c4:37:50 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.100/24 scope global enp7s0
        valid_lft forever preferred_lft forever
    inet6 fe80::fe:95ff:fec4:3750/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@romeo:~$

ubuntu@juliet:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 02:d4:3a:85:9e:bf brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.101/24 scope global enp7s0
        valid_lft forever preferred_lft forever
    inet6 fe80::d4:3aff:fe85:9ebf/64 scope link
        valid_lft forever preferred_lft forever
3: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:99:34:55 brd ff:ff:ff:ff:ff:ff
    inet 10.30.8.30/19 metric 100 brd 10.30.31.255 scope global dynamic enp3s0
        valid_lft 84190sec preferred_lft 84190sec
    inet6 2610:1e0:1700:206:f816:3eff:fe99:3455/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86344sec preferred_lft 14344sec
    inet6 fe80::f816:3eff:fe99:3455/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@juliet:~$

ubuntu@hamlet:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc fq_codel state UP group default qlen 1000
    link/ether fa:16:3e:2c:e5:08 brd ff:ff:ff:ff:ff:ff
    inet 10.30.6.247/19 metric 100 brd 10.30.31.255 scope global dynamic enp3s0
        valid_lft 84196sec preferred_lft 84196sec
    inet6 2610:1e0:1700:206:f816:3eff:fe2c:e508/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86342sec preferred_lft 14342sec
    inet6 fe80::f816:3eff:fe2c:e508/64 scope link
        valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 16:ad:67:79:32:57 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.102/24 scope global enp7s0
        valid_lft forever preferred_lft forever
    inet6 fe80::14ad:67ff:fe79:3257/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@hamlet:~$
```

I use “ip addr” on each node. I notice “enp7s0” is the experiment interface.

Viewing the ARP tables

```
ubuntu@romeo:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@romeo:~$
```

```
ubuntu@juliet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@juliet:~$
```

```
ubuntu@hamlet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@hamlet:~$
```

I use “ip neigh show” and there are no entries for the experiment interface.

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.0.0.102 | grep -oP "(?<=dev )[^\s]+" -w $(hostname -s)-arp.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

```

On another terminal, I ssh into “romeo” to capture network traffic.

```
ubuntu@romeo:~$ ping -c 1 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
64 bytes from 10.0.0.101: icmp_seq=1 ttl=64 time=0.266 ms

--- 10.0.0.101 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.266/0.266/0.266/0.000 ms
ubuntu@romeo:~$
```

Then, I use “ping -c 1 10.0.0.101” to send an ICMP echo request to “juliet”.

```
ubuntu@romeo:~$ ip neigh show
10.0.0.101 dev enp7s0 lladdr 02:d4:3a:85:9e:bf STALE
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
```

This time, there is an entry for the experiment interface “enp7s0”.

```
ubuntu@juliet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
10.0.0.100 dev enp7s0 lladdr 02:fe:95:c4:37:50 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router DELAY
```

```
ubuntu@hamlet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router DELAY
```

Now, “juliet” has an entry for “romeo” and “hamlet” only changes to “DELAY”.

Viewing capture file in terminal

```
ubuntu@romeo:~$ ls
romeo-arp.pcap
ubuntu@romeo:~$ tcpdump -enX -r $(hostname -s)-arp.pcap
reading from file romeo-arp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
18:38:23.240700 02:fe:95:c4:37:50 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 10.0.0.101 tell 10.0.0.100, length 28
    0x0000: 0001 0800 0604 0001 02fe 95c4 3750 0a00 .....7P..
    0x0010: 0064 0000 0000 0000 0a00 0065 .....d.....e
18:38:23.240880 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype ARP (0x0806), length 56: Reply 10.0.0.101 is-at 02:d4:3a:85:9e:bf, length 42
    0x0000: 0001 0800 0604 0002 02d4 3a85 9ebf 0a00 .....
    0x0010: 0065 02fe 95c4 3750 0a00 0064 0000 0000 .....e....7P...d....
    0x0020: 0000 0000 0000 0000 .....
18:38:23.240884 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo request, id 1, seq 1, length 64
    0x0000: 4500 0054 5199 4000 4001 d447 0a00 0064 E..TQ.@.G...d
    0x0010: 0a00 0065 0800 46ec 0001 0001 9f2b 2167 ...e..F.....+!g
    0x0020: 0000 0000 2eac 0300 0000 0000 1011 1213 .....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223 .....!""#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233 $%%'()*+,-./0123
    0x0050: 3435 3637 .....4567
18:38:23.240952 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo reply, id 1, seq 1, length 64
    0x0000: 4500 0054 83ee 0000 4001 e1f2 0a00 0065 E..T....@.....e
    0x0010: 0a00 0064 0000 4eec 0001 0001 9f2b 2167 ...d..N.....+!g
    0x0020: 0000 0000 2eac 0300 0000 0000 1011 1213 .....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223 .....!""#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233 $%%'()*+,-./0123
    0x0050: 3435 3637 .....4567
18:38:28.373475 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype ARP (0x0806), length 56: Request who-has 10.0.0.100 tell 10.0.0.101, length 42
    0x0000: 0001 0800 0604 0001 02d4 3a85 9ebf 0a00 .....
    0x0010: 0065 0000 0000 0000 0a00 0064 0000 0000 .....e.....d....
    0x0020: 0000 0000 0000 0000 .....
18:38:28.373493 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype ARP (0x0806), length 42: Reply 10.0.0.100 is-at 02:fe:95:c4:37:50, length 28
    0x0000: 0001 0800 0604 0002 02fe 95c4 3750 0a00 .....7P..
    0x0010: 0064 02d4 3a85 9ebf 0a00 0065 .....d.....e
18:39:39.540457 16:ad:67:79:32:57 > 33:33:00:00:00:02, ethertype IPv6 (0x86dd), length 70: fe80::14ad:67ff:fe79:3257 > ff02::2: ICMP6, router solicitation, length 16
    0x0000: 6000 0000 0010 3aff fe80 0000 0000 0000 .....
    0x0010: 14ad 67ff fe79 3257 ff02 0000 0000 0000 ..g..yZW.....
    0x0020: 0000 0000 0000 0002 8500 1e33 0000 0000 .....3....
    0x0030: 0101 16ad 6779 3257 .....gyZW
```

I'm able to notice there was an ARP request when "romeo" pinged "juliet", shortly after an ARP reply (response) was received.

The second capture file

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.0.0.102 | grep -oP "(?<=dev )[^ ]+") -w $(hostname -s)-no-arp.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

```
ubuntu@romeo:~$ ping -c 1 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
```

Once again, I captured network traffic on "romeo". Then I pinged once more.

```
ubuntu@romeo:~$ ls
romeo-arp.pcap romeo-no-arp.pcap
ubuntu@romeo:~$ tcpdump -enX -r $(hostname -s)-no-arp.pcap
reading from file romeo-no-arp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
18:58:57.504311 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo request, id 2, seq 1, length 64
    0x0000: 4500 0054 ba36 4000 4001 6ba0 0a00 0064 E..T.6@.k....d
    0x0010: 0a00 0065 0800 b9e0 0002 0001 7130 2167 ...e.....q0!g
    0x0020: 0000 0000 e5b1 0700 0000 0000 1011 1213 .....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223 .....!""#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233 $%%'()*+,-./0123
    0x0050: 3435 3637 .....4567
18:58:57.504478 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo reply, id 2, seq 1, length 64
    0x0000: 4500 0054 fa73 0000 4001 6b6d 0a00 0065 E..T.s..@.km...e
    0x0010: 0a00 0064 0000 c1e0 0002 0001 7130 2167 ...d.....q0!g
    0x0020: 0000 0000 e5b1 0700 0000 0000 1011 1213 .....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223 .....!""#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233 $%%'()*+,-./0123
    0x0050: 3435 3637 .....4567
```

This time, there was no ARP request or response.

Transferring the packet captures

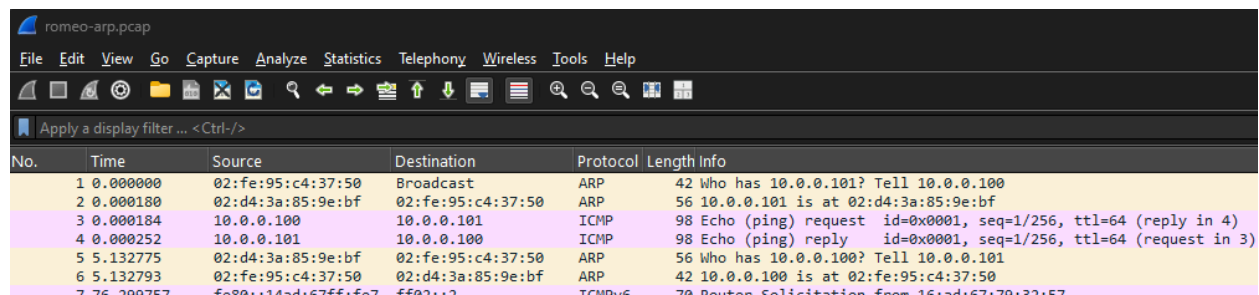
```
fabric@spring:work-16%$ scp ubuntu@2610:1e0:1700:206:f816:3eff:fe2c:e508:/home/ubuntu/romeo-arp.pcap /home/fabric/work/romeo-arp.pcap
ssh: connect to host 0.0.10.50 port 22: Connection timed out
```

```
fabric@spring:work-16%$ scp ubuntu@2610:1e0:1700:206:f816:3eff:fe2c:e508:/home/ubuntu/romeo-no-arp.pcap /home/fabric/work/romeo-no-arp.pcap
ssh: connect to host 0.0.10.50 port 22: Connection timed out
```

I tried using an scp command specifying which file I'd like to transfer and where I'd like to copy it, however, I kept getting "Connection timed out". Then, I tried doing it from inside "romeo", but the problem persisted.

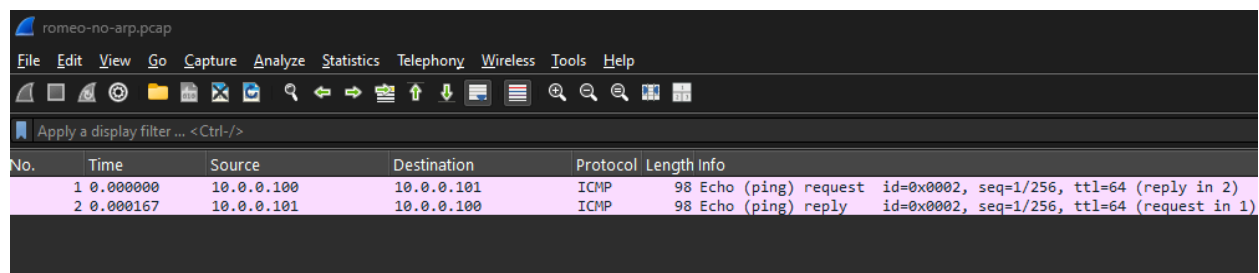
```
ubuntu@romeo:~$ ls
romeo-arp.pcap  romeo-no-arp.pcap
ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-arp.pcap" https://file.io
{"success":true,"status":200,"id":"39ee9b70-9630-11ef-b727-eb20013a92a5","key":"1ln0xt1cdFfK","path":"/","nodeType":"file",
.622Z","modified":"2024-10-29T19:58:54.622Z"}ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-no-arp.pcap" https://file.ic
{"success":true,"status":200,"id":"42040110-9630-11ef-908b-7da7c4195438","key":"Q68h5iwwREih","path":"/","nodeType":"file",
:08.182Z","modified":"2024-10-29T19:59:08.182Z"}ubuntu@romeo:~$
```

So, I ended up using "file.io" like shown in class. This method was successful almost instantly. It provided me with two links: "<https://file.io/1ln0xt1cdFfK>" for "romeo-arp.pcap" & "<https://file.io/Q68h5iwwREih>" for "romeo-no-arp.pcap".



Wireshark capture of romeo-arp.pcap. The capture shows traffic between 10.0.0.100 and 10.0.0.101. The first packet is a broadcast ARP request from 10.0.0.100 to 02:fe:95:c4:37:50. The second packet is an ARP reply from 10.0.0.101 to 02:fe:95:c4:37:50. The third packet is an ICMP Echo (ping) request from 10.0.0.100 to 10.0.0.101. The fourth packet is an ICMP Echo (ping) reply from 10.0.0.101 to 10.0.0.100. The fifth packet is an ARP request from 10.0.0.100 to 02:fe:95:c4:37:50. The sixth packet is an ARP request from 10.0.0.100 to 02:fe:95:c4:37:50. The seventh packet is an ICMPv6 Router Solicitation from 16:ad:67:79:32:57 to ff02::2.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.101? Tell 10.0.0.100
2	0.000180	02:d4:3a:85:9e:bf	02:fe:95:c4:37:50	ARP	56	10.0.0.101 is at 02:d4:3a:85:9e:bf
3	0.000184	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 4)
4	0.000252	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) reply id=0x0001, seq=1/256, ttl=64 (request in 3)
5	5.132775	02:d4:3a:85:9e:bf	02:fe:95:c4:37:50	ARP	56	Who has 10.0.0.100? Tell 10.0.0.101
6	5.132793	02:fe:95:c4:37:50	02:d4:3a:85:9e:bf	ARP	42	10.0.0.100 is at 02:fe:95:c4:37:50
7	76.299757	fe80::14ad:67ff:fe7... ff02::2		ICMPv6	70	Router Solicitation from 16:ad:67:79:32:57



Wireshark capture of romeo-no-arp.pcap. The capture shows traffic between 10.0.0.100 and 10.0.0.101. The first packet is an ICMP Echo (ping) request from 10.0.0.100 to 10.0.0.101. The second packet is an ICMP Echo (ping) reply from 10.0.0.101 to 10.0.0.100.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (reply in 2)
2	0.000167	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=64 (request in 1)

I was able to download and view both files in Wireshark with no issues.

Verifying neighbor reachability

```
ubuntu@romeo:~$ ip neigh show
10.0.0.101 dev enp7s0 lladdr 02:d4:3a:85:9e:bf STALE
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
fe80::eab2:65ff:fe0b:fe11 dev enp3s0 lladdr e8:b2:65:0b:fe:11 router STALE
```

I use “ip neigh show” on “romeo” again. Notice that after some time passed, the entry for “juliet” has gone “STALE”.

Continuously monitoring the state of the ARP table

```
ubuntu@romeo:~$ watch -n 0.1 ip neigh show dev $(ip route get 10.0.0.102 | grep -oP "(?<=dev )[^ ]+")
```

```
Every 0.1s: ip neigh show dev enp7s0
10.0.0.101 lladdr 02:d4:3a:85:9e:bf STALE
```

On another “romeo” terminal, I use a command to monitor the state of the ARP table for the experiment interface.

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.0.0.102 | grep -oP "(?<=dev )[^ ]+") -w $(hostname -s)-poll-arp.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[]
```

Then, I run “sudo tcpdump -i \$(ip route get 10.0.0.102 | grep -oP "(?<=dev)[^]+") -w \$(hostname -s)-poll-arp.pcap” a third “romeo” terminal.

```
ubuntu@romeo:~$ ping -c 1 10.0.0.101
PING 10.0.0.101 (10.0.0.101) 56(84) bytes of data.
64 bytes from 10.0.0.101: icmp_seq=1 ttl=64 time=0.175 ms

--- 10.0.0.101 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.175/0.175/0.175/0.000 ms
```

Lastly, I use “ping -c 1 10.0.0.101” to ping “juliet”. Right after this, I was able to notice a change in the terminal monitoring the state of the ARP table. I wasn’t able to capture it, but it went from “STALE” to “DELAY” to “REACHABLE”.

Viewing the new capture file

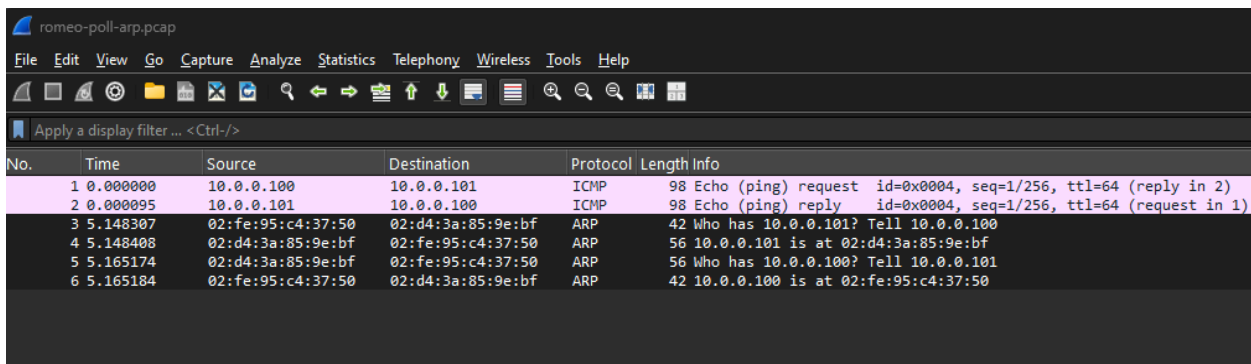
```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.0.0.102 | grep -oP "(?<=dev)[^ ]+") -w $(hostname -s)-poll-arp.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C6 packets captured
0 packets received by filter
0 packets dropped by kernel
ubuntu@romeo:~$ tcpdump -enX -r $(hostname -s)-poll-arp.pcap
reading from file romeo-poll-arp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
20:25:43.925974 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype IPv4 (0x0800), length 98: 10.0.0.100 > 10.0.0.101: ICMP echo request, id 4, seq 1, length 64
    0x0000: 4500 0054 851d 4000 4001 a0c3 0a00 0064  E..T...@.....d
    0x0010: 0a00 0065 0800 385b 0004 0001 c744 2167  ...e..8[.....D!g
    0x0020: 0000 0000 0a21 0e00 0000 0000 1011 1213  .....!.....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223  .....!"#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233  $%&'()*+,-./0123
    0x0050: 3435 3637 4567
20:25:43.926069 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype IPv4 (0x0800), length 98: 10.0.0.101 > 10.0.0.100: ICMP echo reply, id 4, seq 1, length 64
    0x0000: 4500 0054 5520 0000 4001 10c1 0a00 0065  E..TU...@.....e
    0x0010: 0a00 0064 0000 405b 0004 0001 c744 2167  ...d..@[.....D!g
    0x0020: 0000 0000 0a21 0e00 0000 0000 1011 1213  .....!.....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223  .....!"#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233  $%&'()*+,-./0123
    0x0050: 3435 3637 4567
20:25:49.074281 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype ARP (0x0806), length 42: Request who-has 10.0.0.101 tell 10.0.0.100, length 28
    0x0000: 0001 0800 0604 0001 02fe 95c4 3750 0a00  .....7P..
    0x0010: 0064 0000 0000 0000 0a00 0065 .....d.....e
20:25:49.074382 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype ARP (0x0806), length 56: Reply 10.0.0.101 is-at 02:d4:3a:85:9e:bf, length 42
    0x0000: 0001 0800 0604 0002 02d4 3a85 9ebf 0a00  .....!.....
    0x0010: 0065 02fe 95c4 3750 0a00 0064 0000 0000  .e....7P...d....
    0x0020: 0000 0000 0000 0000 0000 .....
20:25:49.091148 02:d4:3a:85:9e:bf > 02:fe:95:c4:37:50, ethertype ARP (0x0806), length 56: Request who-has 10.0.0.100 tell 10.0.0.101, length 42
    0x0000: 0001 0800 0604 0001 02d4 3a85 9ebf 0a00  .....!.....
    0x0010: 0065 0000 0000 0000 0a00 0064 0000 0000  .e.....d....
    0x0020: 0000 0000 0000 0000 0000 .....
20:25:49.091158 02:fe:95:c4:37:50 > 02:d4:3a:85:9e:bf, ethertype ARP (0x0806), length 42: Reply 10.0.0.100 is-at 02:fe:95:c4:37:50, length 28
    0x0000: 0001 0800 0604 0002 02fe 95c4 3750 0a00  .....7P..
    0x0010: 0064 02d4 3a85 9ebf 0a00 0065 .....d.....e
ubuntu@romeo:~$
```

Here, I notice 2 ARP requests and responses.

Transferring “romeo-poll-arp.pcap” using Curl

```
ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-poll-arp.pcap" https://file.io
{"success":true,"status":200,"id":"90bc2590-9634-11ef-ab07-0b670c17429b","key":"uNqKROC3Mleb","path":"/","nodeType":"file",
29:58.237Z","modified":"2024-10-29T20:29:58.237Z"}ubuntu@romeo:~$
```

I use “file.io” to download the file at: <https://file.io/uNqKROC3Mleb>.”



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.100	10.0.0.101	ICMP	98	Echo (ping) request id=0x0004, seq=1/256, ttl=64 (reply in 2)
2	0.000095	10.0.0.101	10.0.0.100	ICMP	98	Echo (ping) reply id=0x0004, seq=1/256, ttl=64 (request in 1)
3	5.148307	02:fe:95:c4:37:50	02:d4:3a:85:9e:bf	ARP	42	Who has 10.0.0.101? Tell 10.0.0.100
4	5.148408	02:d4:3a:85:9e:bf	02:fe:95:c4:37:50	ARP	56	10.0.0.101 is at 02:d4:3a:85:9e:bf
5	5.165174	02:d4:3a:85:9e:bf	02:fe:95:c4:37:50	ARP	56	Who has 10.0.0.100? Tell 10.0.0.101
6	5.165184	02:fe:95:c4:37:50	02:d4:3a:85:9e:bf	ARP	42	10.0.0.100 is at 02:fe:95:c4:37:50

I’m also able to view it on Wireshark.

ARP for a non-existent host

For this, I'll be using 4 different "romeo" terminals.

```
Every 0.1s: ip neigh show dev enp7s0
```

```
10.0.0.101 lladdr 02:d4:3a:85:9e:bf STALE
```

In the first, I use the command "watch -n 0.1 ip neigh show dev \$(ip route get 10.0.0.102 | grep -oP "(?<=dev)[^]+)" to continuously monitor the state.

```
ubuntu@romeo:~$ sudo tcpdump -i $(ip route get 10.0.0.102 | grep -oP "(?<=dev )[^ ]+") -w $(hostname -s)-eth-nonexistent.pcap
tcpdump: listening on enp7s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[]
```

Then, in the second terminal I use "sudo tcpdump -i \$(ip route get 10.0.0.102 | grep -oP "(?<=dev)[^]+") -w \$(hostname -s)-eth-nonexistent.pcap" to capture network traffic.

```
ubuntu@romeo:~$ sudo tcpdump -i lo -w $(hostname -s)-lo-nonexistent.pcap icmp
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[]
```

In the 3rd terminal, I use "sudo tcpdump -i lo -w \$(hostname -s)-lo-nonexistent.pcap icmp" to capture ICMP traffic on the loopback interface.

```
ubuntu@romeo:~$ ping -c 1 10.0.0.200
PING 10.0.0.200 (10.0.0.200) 56(84) bytes of data.
From 10.0.0.100 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.200 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

ubuntu@romeo:~$ []
```

Lastly, I use "ping -c 1 10.0.0.200" to ping an IP that doesn't belong to any hosts in the network configuration. This is evident through the "100% packet loss".

Viewing results

```
Every 0.1s: ip neigh show dev enp7s0

10.0.0.200 FAILED
10.0.0.101 lladdr 02:d4:3a:85:9e:bf REACHABLE
```

Notice this time, there is a “FAILED” status next to the non-existent host.

```
ubuntu@romeo:~$ tcpdump -enX -r $(hostname -s)-eth-nonexistent.pcap
reading from file romeo-eth-nonexistent.pcap, link-type EN10MB (Ethernet), snapshot length 262144
20:47:26.982835 02:fe:95:c4:37:50 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 10.0.0.200 tell 10.0.0.100, length 28
0x0000: 0001 0800 0604 0001 02fe 95c4 3750 0a00 .....7P..
0x0010: 0064 0000 0000 0000 0a00 00c8 .....d.....
20:47:27.986294 02:fe:95:c4:37:50 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 10.0.0.200 tell 10.0.0.100, length 28
0x0000: 0001 0800 0604 0001 02fe 95c4 3750 0a00 .....7P..
0x0010: 0064 0000 0000 0000 0a00 00c8 .....d.....
20:47:29.010281 02:fe:95:c4:37:50 > ff:ff:ff:ff:ff:ff, ethertype ARP (0x0806), length 42: Request who-has 10.0.0.200 tell 10.0.0.100, length 28
0x0000: 0001 0800 0604 0001 02fe 95c4 3750 0a00 .....7P..
0x0010: 0064 0000 0000 0000 0a00 00c8 .....d.....
20:48:23.843890 02:d4:3a:85:9e:bf > 33:33:00:00:00:02, ethertype IPv6 (0x86dd), length 70: fe80::d4:3aff:fe85:9ebf > ff02::2: ICMPv6, router solicitation, length 16
0x0000: 6000 0000 0010 3aff fe80 0000 0000 0000 .....
0x0010: 00d4 3aff fe85 9ebf ff02 0000 0000 0000 .....
0x0020: 0000 0000 0000 0002 8500 c6fc 0000 0000 .....
0x0030: 0101 02d4 3a85 9ebf .....

```

The “romeo-eth-nonexistent.pcap” file shows only ARP requests this time.

```
ubuntu@romeo:~$ tcpdump -enX -r $(hostname -s)-lo-nonexistent.pcap
reading from file romeo-lo-nonexistent.pcap, link-type EN10MB (Ethernet), snapshot length 262144
20:47:30.034288 00:00:00:00:00:00 > 00:00:00:00:00:00, ethertype IPv4 (0x0800), length 126: 10.0.0.100 > 10.0.0.100: ICMP host 10.0.0.200 unreachable, length 92
0x0000: 45c0 0070 6197 0000 4001 036f 0a00 0064 E..pa...@...o...d
0x0010: 0a00 0064 0301 fcfe 0000 0000 4500 0054 ...d.....E..T
0x0020: 6205 4000 4001 c378 0a00 0064 0a00 00c8 b.@...x...d....
0x0030: 0800 0677 0005 0001 de49 2167 0000 0000 ...w.....!lg....
0x0040: 24ff 0e00 0000 0000 1011 1213 1415 1617 $.
0x0050: 1819 1a1b 1c1d 1e1f 2021 2223 2425 2627 .....!""$%&'
0x0060: 2829 2a2b 2c2d 2e2f 3031 3233 3435 3637 ()+,-./01234567

```

In the “romeo-lo-nonexistent.pcap” file notice it says “ICMP host 10.0.0.200 unreachable”.

Transferring and viewing the remaining capture files

```
ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-lo-nonexistent.pcap" https://file.io
{"success":true,"status":200,"id":"84f18350-9638-11ef-a8e1-8749aa288cfc","key":"CZKe9sLl91VT","path":"/","nodeType":"file",
29T20:58:16.423Z","modified":"2024-10-29T20:58:16.423Z"}ubuntu@romeo:~$ curl -F "file=@/home/ubuntu/romeo-eth-nonexistent.p
{"success":true,"status":200,"id":"91505270-9638-11ef-81cf-1d869a620910","key":"dsgxznOjAgaw","path":"/","nodeType":"file",
-29T20:58:37.199Z","modified":"2024-10-29T20:58:37.199Z"}ubuntu@romeo:~$
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
2	1.003459	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
3	2.027446	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
4	56.861055	fe80::d4:3aff:fe85:9e...	ff02::2	ICMPv6	70	Router Solicitation from 02:d4:3a:85:9e:bf

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.100	10.0.0.100	ICMP	126	Destination unreachable (Host unreachable)

Once again, I used “file.io” to download the files and view them in Wireshark.

Exercises

Basic ARP request and response

1. What is the target IP address in the ARP request?

```
1 0.000000 02:fe:95:c4:37:50 Broadcast ARP 42 Who has 10.0.0.101? Tell 10.0.0.100
```

The target IP would be “10.0.0.101” (juliet).

2. At the MAC layer, what is the destination Ethernet address of the frame carrying the ARP request? Why?

```
Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
Ethernet II, Src: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  ▶ Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  ▶ Source: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50)
  Type: ARP (0x0806)
  [Stream index: 0]
```

The destination Ethernet address for an ARP request frame is “ff:ff:ff:ff:ff:ff” (broadcast). This is because ARP requests are sent to all devices on the local network, allowing the device with the target IP to respond.

3. What is the frame type field in the Ethernet frame?

```
Type: ARP (0x0806)
```

The frame type field for ARP in Ethernet frames is typically 0x0806, which signifies the ARP protocol, allowing network devices to identify the traffic type.

4. Of the three hosts on your network segment, which host sends the ARP reply? Why?

```
2 0.000180 02:d4:3a:85:9e:bf 02:fe:95:c4:37:50 ARP 56 10.0.0.101 is at 02:d4:3a:85:9e:bf
```

It’s “juliet” that responds with an ARP reply, since it recognizes the requested IP address as its own, providing its MAC address for resolution.

5. When an ARP request and ARP reply appear on a network segment, which hosts on the network segment will add the target of the ARP request to their ARP table? Which hosts on the network segment will add the sender of the ARP request to their ARP table? Explain in general, as well as for the specific case of this network.

romeo:

```
ubuntu@romeo:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@romeo:~$

ubuntu@romeo:~$ ip neigh show
10.0.0.101 dev enp7s0 lladdr 02:d4:3a:85:9e:bf STALE
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
```

juliet:

```
ubuntu@juliet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@juliet:~$

ubuntu@juliet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
10.0.0.100 dev enp7s0 lladdr 02:fe:95:c4:37:50 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router DELAY
```

hamlet:

```
ubuntu@hamlet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router REACHABLE
ubuntu@hamlet:~$

ubuntu@hamlet:~$ ip neigh show
10.30.6.11 dev enp3s0 lladdr fa:16:3e:a8:a1:c4 STALE
fe80::f816:3eff:fec1:c8da dev enp3s0 lladdr fa:16:3e:c1:c8:da router DELAY
```

Typically, the host that initiated the ARP request will add the target's IP and MAC to its ARP table upon receiving the reply. In this network, other hosts can also passively observe the ARP broadcast and update their tables if they recognize the sender's or target's address. Notice, "hamlet" did not add "juliet" or "romeo" to its table, but "romeo" and "juliet" now appear on each other's.

Verifying neighbor reachability

1. What is the target IP address in the ARP request?

1	0.000000	10.0.0.100	10.0.0.101	ICMP	98 Echo (ping) request	id=0x0004, seq=1/256, ttl=64 (reply in 2)
2	0.000095	10.0.0.101	10.0.0.100	ICMP	98 Echo (ping) reply	id=0x0004, seq=1/256, ttl=64 (request in 1)
3	5.148307	02:fe:95:c4:37:50	02:d4:3a:85:9e:bf	ARP	42 Who has 10.0.0.101? Tell 10.0.0.100	
4	5.148408	02:d4:3a:85:9e:bf	02:fe:95:c4:37:50	ARP	56 10.0.0.101 is at 02:d4:3a:85:9e:bf	

The target was also “juliet”’s “10.0.0.101” IP address.

2. At the MAC layer, what is the destination Ethernet address of the frame carrying the ARP request? Why?

```
Ethernet II, Src: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50),  
  Destination: 02:d4:3a:85:9e:bf (02:d4:3a:85:9e:bf)  
  Source: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50)  
  Type: ARP (0x0806)  
  [Stream index: 0]
```

The destination Ethernet address is now “02:d4:3a:85:9e:bf” (juliet). This is because the status is only “STALE”, and “romeo” still knows who “10.0.0.101” is.

3. How is this ARP request different from the one sent in the previous section – both in its content, and in its purpose?

```
Ethernet II, Src: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50), Dst: 02:d4:3a:85:9e:bf (02:d4:3a:85:9e:bf)  
  Destination: 02:d4:3a:85:9e:bf (02:d4:3a:85:9e:bf)  
    ... ..1. .... = LG bit: Locally administered address (this is NOT the factory)  
    ... ..0. .... = IG bit: Individual address (unicast)  
  Source: 02:fe:95:c4:37:50 (02:fe:95:c4:37:50)  
    ... ..1. .... = LG bit: Locally administered address (this is NOT the factory)  
    ... ..0. .... = IG bit: Individual address (unicast)  
  Type: ARP (0x0806)  
  [Stream index: 0]
```

In the first ARP request, “romeo” didn’t have any idea who “10.0.0.101” belonged to. In this request, “romeo” has received a reply from the target IP and has added “juliet” to the ARP table. This is why the destination shows the known “juliet” MAC address. The purpose of this second request is to receive up to date information. Meaning, “romeo” wants to verify that this IP address is still reachable at the MAC address it has on file.

ARP for a non-existent host

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
2	1.003459	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
3	2.027446	02:fe:95:c4:37:50	Broadcast	ARP	42	Who has 10.0.0.200? Tell 10.0.0.100
4	56.861055	fe80::d4:3aff:fe85:...	ff02::2	ICMPv6	70	Router Solicitation from 02:d4:3a:85:9e:bf

1. In the previous exercise, after sending an ARP request and receiving a reply, "romeo" sends an ICMP echo request. In this exercise, is an ICMP echo request ever sent? Why or why not?

No, because an ICMP echo request is not sent if the ARP request does not receive a response. The ICMP echo request relies on address resolution, so in the absence of an ARP reply, it is unable to proceed.

2. From the tcpdump output, describe how the ARP timeout and retransmission were performed. How many attempts were made to resolve a non-existing IP address? How much time separates each attempt?

There were 3 attempts made. They were sent in 1 second intervals. After this, the ARP entry timed out, marking the address as unreachable in the ARP table.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.100	10.0.0.100	ICMP	126	Destination unreachable (Host unreachable)

3. What is the value of the ICMP type field and the ICMP code field in the ICMP message?

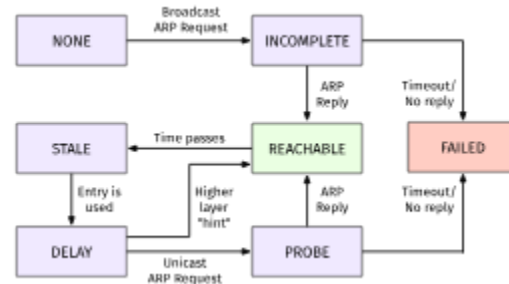
```
Internet Control Message Protocol
Type: 3 (Destination unreachable)
Code: 1 (Host unreachable)
Checksum: 0xfcfe [correct]
[Checksum Status: Good]
Unused: 00000000
```

There was a Type 3 (Destination Unreachable) and Code 1 (Host Unreachable) in the ICMP message, meaning the target was unreachable.

4. Why does this message appear on the loopback interface, and not on the Ethernet segment?

This message appears on the loopback interface because it is a locally generated diagnostic message. When ARP fails, the system produces an internal notification of unreachable status without sending it over the network, since there is no MAC address resolved for external communication.

ARP State Machine



1. Transition from "NONE" to "INCOMPLETE" and "REACHABLE"

Transition from NONE to INCOMPLETE: A broadcast ARP request is sent to determine the MAC address associated with the target IP address. This ARP request is broadcasted because the sender does not yet know the target's MAC address. Transition to REACHABLE: When an ARP reply is received from the target, providing the requested MAC address, the state transitions to REACHABLE. This ARP reply includes the MAC address of the target, which allows the sender to update its ARP table with the IP-to-MAC mapping.

2. Transition from "DELAY" to "PROBE" and "REACHABLE"

Transition from DELAY to PROBE: In the DELAY state, the ARP entry is considered stale, meaning it has not been recently confirmed as active. When it moves to PROBE, a unicast ARP request is sent to the target IP to confirm the entry's

validity without broadcasting to the entire network. Transition to REACHABLE: Message Received: If the target responds to this unicast ARP request, the state is updated to REACHABLE. This indicates that the target IP is still accessible, and the ARP entry is considered fresh.

3. Transition to "FAILED" for an Entry

Transition from NONE to INCOMPLETE: Similar to the first transition, a broadcast ARP request is sent. The system attempts to resolve the IP address by broadcasting to the network. Transition to FAILED: If there is no response after multiple ARP attempts, the entry transitions to FAILED, meaning that the IP address could not be resolved. At this point, an ICMP "Destination Unreachable" message may be generated locally (on the loopback interface) as a diagnostic to indicate that the target IP is unreachable. This message does not go over the Ethernet network since it's an internal notification informing the system that the target IP address could not be resolved due to the lack of an ARP reply.

Deleting the slice

Slice	
ID	75d4395b-e43e-4b77-ac88-a9de517b0738
Name	Wireshark_Stephanie_Salgadossalg03_0000228084
Lease Expiration (UTC)	2024-10-30 17:42:07 +0000
Lease Start (UTC)	2024-10-29 17:42:07 +0000
Project ID	a70de2f5-9e12-4b6b-b412-0ae1a2c553b0
State	Dead

Lastly, I can return to "CLab7.ipynb" to delete the slice I used for this lab.

Summary

In this lab, I used FABRIC to simulate an Address Resolution Protocol (ARP) experiment. After reserving resources from "EDUKY," I configured the environment and established network segments. I conducted basic ARP requests by pinging "juliet" from "romeo" and captured the resulting network traffic, noting the ARP requests and replies. I also experimented with pinging a non-existent IP address to observe the "FAILED" status and captured relevant traffic. I was able to monitor the ARP table's state and analyzed how the ARP state machine transitions occur based on responses. Additionally, I was able to use "file.io" to download the capture files generated throughout this lab. Overall, the lab deepened my understanding of ARP.